arXiv:2209.13085v2 [cs.LG] 5 Mar 2025

Defining and Characterizing Reward Hacking

Joar Skalse[*]
University of Oxford

Nikolaus H. R. Howe
Mila, Université de Montréal

Dmitrii Krasheninnikov
University of Cambridge

David Krueger[*]
University of Cambridge

Abstract

We provide the first formal definition of **reward hacking**, a phenomenon where optimizing an imperfect proxy reward function, $\tilde{\mathcal{R}}$, leads to poor performance according to the true reward function, $\mathcal{R}$. We say that a proxy is **unhackable** if increasing the expected proxy return can never decrease the expected true return. Intuitively, it might be possible to create an unhackable proxy by leaving some terms out of the reward function (making it "narrower") or overlooking fine-grained distinctions between roughly equivalent outcomes, but we show this is usually not the case. A key insight is that the linearity of reward (in state-action visit counts) makes unhackability a very strong condition. In particular, for the set of all stochastic policies, two reward functions can only be unhackable if one of them is constant. We thus turn our attention to deterministic policies and finite sets of stochastic policies, where non-trivial unhackable pairs always exist, and establish necessary and sufficient conditions for the existence of simplifications, an important special case of unhackability. Our results reveal a tension between using reward functions to specify narrow tasks and aligning AI systems with human values.

1 Introduction

It is well known that optimising a proxy can lead to unintended outcomes: a boat spins in circles collecting "powerups" instead of following the race track in a racing game (Clark and Amodei, 2016); an evolved circuit listens in on radio signals from nearby computers' oscillators instead of building its own (Bird and Layzell, 2002); universities reject the most qualified applicants in order to appear more selective and boost their ratings (Golden, 2001). In the context of reinforcement learning (RL), such failures are called **reward hacking**.

For AI systems that take actions in safety-critical real world environments such as autonomous vehicles, algorithmic trading, or content recommendation systems, these unintended outcomes can be catastrophic. This makes it crucial to align autonomous AI systems with their users' intentions. Precisely specifying which behaviours are or are not desirable is challenging, however. One approach to this specification problem is to learn an approximation of the true reward function (Ng et al., 2000; Ziebart, 2010; Leike et al., 2018). Optimizing a learned proxy reward can be dangerous, however; for instance, it might overlook side-effects (Krakovna et al., 2018; Turner et al., 2019) or encourage power-seeking (Turner et al., 2021) behavior. This raises the question motivating our work: When is it safe to optimise a proxy?

To begin to answer this question, we consider a somewhat simpler one: When *could* optimising a proxy lead to worse behaviour? "Optimising", in this context, does not refer to finding a global, or even local, optimum, but rather running a search process, such as stochastic gradient descent

[*]Equal contribution. Correspondence to: `joar.mvs@gmail.com, david.scott.krueger@gmail.com`

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

(SGD), that yields a sequence of candidate policies, and tends to move towards policies with higher (proxy) reward. We make no assumptions about the path through policy space that optimisation takes.[1] Instead, we ask whether there is *any* way in which improving a policy according to the proxy could make the policy worse according to the true reward; this is equivalent to asking if there exists a pair of policies π_1, π_2 where the proxy prefers π_1, but the true reward function prefers π_2. When this is the case, we refer to this pair of true reward function and proxy reward function as **hackable**.

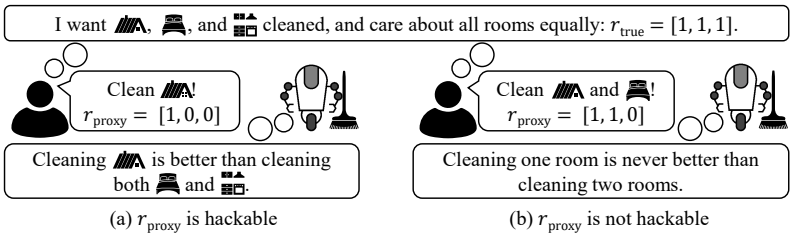
Given the strictness of our definition, it is not immediately apparent that any non-trivial examples of unhackable reward function pairs exist. And indeed, if we consider the set of all stochastic policies, they do not (Section 5.1). However, restricting ourselves to *any* finite set of policies guarantees at least one non-trivial unhackable pair (Section 5.2).

Intuitively, we might expect the proxy to be a "simpler" version of the true reward function. Noting that the definition of unhackability is symmetric, we introduce the asymmetric special case of **simplification**, and arrive at similar theoretical results for this notion.[2] In the process, and through examples, we show that seemingly natural ways of simplifying reward functions often fail to produce simplifications in our formal sense, and in fact fail to rule out the potential for reward hacking.

We conclude with a discussion of the implications and limitations of our work. Briefly, our work suggests that a proxy reward function must satisfy demanding standards in order for it to be safe to optimize. This in turn implies that the reward functions learned by methods such as reward modeling and inverse RL are perhaps best viewed as auxiliaries to policy learning, rather than specifications that should be optimized. This conclusion is weakened, however, by the conservativeness of our chosen definitions; future work should explore when hackable proxies can be shown to be safe in a probabilistic or approximate sense, or when subject to only limited optimization.

2 Example: Cleaning Robot

Consider a household robot tasked with cleaning a house with three rooms: Attic, Bedroom, and Kitchen. The robot's (deterministic) policy is a vector indicating which rooms it cleans: $\pi = [\pi_1, \pi_2, \pi_3] \in \{0, 1\}^3$. The robot receives a (non-negative) reward of r_1, r_2, r_3 for cleaning the attic, bedroom, and kitchen, respectively, and the total reward is given by $J(\pi) = \pi \cdot r$. For example, if $r = [1, 2, 3]$ and the robot cleans the attic and the kitchen, it receives a reward of $1 + 3 = 4$.



(a) r_{proxy} is hackable (b) r_{proxy} is not hackable

Figure 1: An illustration of hackable and unhackable proxy rewards arising from overlooking rewarding features. A human wants their house cleaned. In (a), the robot draws an incorrect conclusion because of the proxy; this could lead to hacking. In (b), no such hacking can occur: the proxy is unhackable.

At least two ideas come to mind when thinking about "simplifying" a reward function. The first one is *overlooking rewarding features*: suppose the true reward is equal for all the rooms, $r_{\text{true}} = [1, 1, 1]$, but we only ask the robot to clean the attic and bedroom, $r_{\text{proxy}} = [1, 1, 0]$. In this case, r_{proxy} and r_{true} are unhackable. However, if we ask the robot to only clean the attic, $r_{\text{proxy}} = [1, 0, 0]$, this is hackable with respect to r_{true}. To see this, note that according to r_{proxy} cleaning the attic ($J_{\text{proxy}} = 1$) is better than cleaning the bedroom and the kitchen ($J_{\text{proxy}} = 0$). Yet, r_{true} says that cleaning the attic ($J_{\text{true}} = 1$) is worse than cleaning the bedroom and the kitchen ($J_{\text{true}} = 2$). This situation is illustrated in Figure 1.

[1] This assumption – although conservative – is reasonable because optimisation in state-of-the-art deep RL methods is poorly understood and results are often highly stochastic and suboptimal.

[2] See Section 4.2 for formal definitions.

The second seemingly natural way to simplify a reward function is *overlooking fine details*: suppose $r_{\text{true}} = [1, 1.5, 2]$, and we ask the robot to clean all the rooms, $r_{\text{proxy}} = [1, 1, 1]$. For these values, the proxy and true reward are unhackable. However, with a slightly less balanced true reward function such as $r_{\text{true}} = [1, 1.5, 3]$ the proxy does lead to hacking, since the robot would falsely calculate that it's better to clean the attic and the bedroom than the kitchen alone.

These two examples illustrate that while simplification of reward functions is sometimes possible, attempts at simplification can easily lead to reward hacking. Intuitively, omitting/overlooking details is okay so long as all these details are not as important together as any of the details that we do share. In general, it is not obvious what the proxy must look like to avoid reward hacking, suggesting we should take great care when using proxies. For this specific environment, a proxy and a true reward are hackable exactly when there are two sets of rooms S_1, S_2 such that the true reward gives strictly higher value to cleaning S_1 than it does to cleaning S_2, and the proxy says the opposite: $J_1(S_1) > J_1(S_2)$ & $J_2(S_1) < J_2(S_2)$. For a proof of this statement, see Appendix D.2.1.

3 Related Work

While we are the first to define hackability, we are far from the first to study specification hacking. The observation that optimizing proxy metrics tends to lead to perverse instantiations is often called "Goodhart's Law", and is attributed to Goodhart (1975). Manheim and Garrabrant (2018) provide a list of four mechanisms underlying this observation.

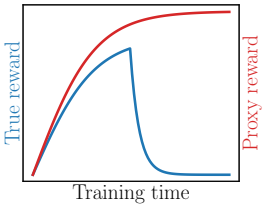


Figure 2: An illustration of reward hacking when optimizing a hackable proxy. The true reward first increases and then drops off, while the proxy reward continues to increase.

Examples of such unintended behavior abound in both RL and other areas of AI; Krakovna et al. (2020) provide an extensive list. Notable recent instances include a robot positioning itself between the camera and the object it is supposed to grasp in a way that tricks the reward model (Amodei et al., 2017), the previously mentioned boat race example (Clark and Amodei, 2016), and a multitude of examples of reward model hacking in Atari (Ibarz et al., 2018). Reward hacking can occur suddenly. Ibarz et al. (2018) and Pan et al. (2022) showcase plots similar to one in Figure 2, where optimizing the proxy (either a learned reward model or a hand-specified reward function) first leads to both proxy and true rewards increasing, and then to a sudden phase transition where the true reward collapses while the proxy continues going up.

Note that not all of these examples correspond to optimal behavior according to the proxy. Indeed, convergence to suboptimal policies is a well-known issue in RL (Thrun and Schwartz, 1993). As a consequence, improving optimization often leads to unexpected, qualitative changes in behavior. For instance, Zhang et al. (2021) demonstrate a novel cartwheeling behavior in the widely studied Half-Cheetah environment that exceeds previous performance so greatly that it breaks the simulator. The unpredictability of RL optimization is a key motivation for our definition of hackability, since we cannot assume that agents will find an optimal policy. Neither can we rule out the possibility of sudden improvements in proxy reward and corresponding qualitative changes in behavior. Unhackability could provide confidence that reward hacking will not occur despite these challenges.

Despite the prevalence and potential severity of reward hacking, to our knowledge Pan et al. (2022) provide the first peer-reviewed work that focuses specifically on it, although Everitt et al. (2017) tackle the closely related issue of reward corruption. The work of Pan et al. (2022) is purely empirical; they manually construct proxy rewards for several diverse environments, and evaluate whether optimizing these proxies leads to reward hacking; in 5 out of 9 of their settings, it does. In another closely related work, Zhuang and Hadfield-Menell (2020) examine what happens when the proxy reward function depends on a strict subset of features relevant for the true reward. They show that optimizing the proxy reward can lead to arbitrarily low true reward under suitable assumptions. This can be seen as a seemingly valid simplification of the true reward that turns out to be (highly) hackable. While their result only applies to environments with decreasing marginal utility and increasing opportunity cost, we demonstrate hackability is an issue in arbitrary MDPs.